

ODABRANA POGLAVLJA IZ OPERATIVNIH SISTEMA

Odgovori na pitanja za usmeni dio ispita

Univerzitet u Banjaluci

Elektrotehnički fakultet

Vanja Novaković

Napomena: Ova skripta nastala je isključivo kako bi nadomjestila nedostatak neke druge cjelovite literature na maternjem jeziku, te kao takva nije reprezentacija djela bez nedostataka. Svaka postojeća greška je ili rezultat nepažnje ili rezultat neznanja. Može predstavljati svojevrstan "open source" materijal kojem svako može da doprinese.

1. Sekvenca pokretanja operativnog sistema ^[1]

U računarstvu, *booting* predstavlja pokretanje računara. Može se inicirati hardverski (pritiskom na dugme) ili softverski (zadavanjem komande). U modernim operativnim sistemima, princip funkcionisanja je sljedeći: kada se računar ugasi, sve što je neophodno za njegovo ponovno pokretanje, smješteno je u trajnu memoriju. Nakon uključivanja računara, operativni sistem ili program za njegovo učitavanje se ne nalaze u RAM memoriji. Prilikom pokretanja, računar prvo izvrši mali dio koda koji je smješten u ROM memoriji zajedno sa potrebnim podacima. Taj dio koda predstavlja osnovu za pristup RAM memoriji.

Mali program koji započinje ovaj proces ili sekvencu pokretanja naziva se *bootstrap loader*, *bootstrap* ili *boot loader*¹. Njegov jedini zadatak je učitavanje drugih programa koji se onda izvršavaju iz RAM-a te podataka koji su potrebni tim programima. U praksi se pretežno koriste *multiple-stage boot loaderi* (višefazni boot-loaderi), kod kojih se lančano učitavaju program za programom, tako da je složenost narednog programa veća od složenosti prethodno učitano. Kod nekih računara, nakon primanja signala za pokretanje, učitava se veoma mali broj fiksnih instrukcija koje se smještaju na specifične lokacije u memoriji, inicijalizuje se (barem) jedan CPU, a zatim se taj CPU usmjerava na izvršavanje prethodno pomenutih instrukcija. Manji računari često koriste manje fleksibilne, ali automatizovanije *boot loader* mehanizme kako bi se računar pokrenuo brzo i sa unaprijed utvrđenom softverskom konfiguracijom. Kod većine desktop računara, bootstrapping proces počinje tako što CPU izvršava kod koji se nalazi na predefinisanoj adresi u ROM-u (npr. UEFI, BIOS..). Ovaj kod sadrži osnovne funkcionalnosti neophodne za pretraživanje uređaja koji ispunjavaju uslove potrebne za učešće u procesu pokretanja operativnog sistema i učitava mali program sa posebne sekcije (uglavnom boot sektor) uređaja koji najviše obećava.

Boot loader prve faze (*First-stage boot loader*)

Programi za učitavanje OS (*boot loaderi*) nailaze na razna ograničenja, naročito po pitanju veličine: npr. na IBM i kompatibilnim računarima, sektor za pokretanje (*boot sector*) treba da radi na samo 32KB (kasnije prošireno na 64KB) systemske memorije i koristi samo instrukcije koje podržavaju originalni 8088/8086 procesori. FSB² (*first-stage boot loader*) lociran na HDD-u i prenosivim drajvovima mora stati u prvih 446 bajtova MBR-a (Master Boot Record) kako bi ostalo prostora za podrazumijevanu 64-bitnu tabelu particionisanja za 4 particiona unosa i 2-bajtnim potpisom za pokretanje (*boot signature*) kojeg BIOS zathjeva za ispravno pokretanje OS-a. Ponekad mora da zauzima čak i manje prostora – u složenijim slučajevima, npr. kada postoje više od 4 particiona unosa (do 16 sa po 16 bajtova), potpisom za disk (6 bajtova), vremenskom oznakom za disk (6 bajtova), AAP (Advanced Active Partition – 18 bajtova) i sl.

U floppy i superfloppy VBR³, do 51 bajta zauzima BIOS Parameter Block na FAT12 i FAT16, dok FAT32 zahtijeva čak 79 bajtova, tako da preostaju samo 432 bajta za boot loader, ako pretpostavimo da je veličina sektora 512 bajtova. Microsoft-ovi sektori za pokretanje su stoga tradicionalno nametali određena ograničenja za proces pokretanja, npr. fajl za pokretanje morao je biti lociran na fiksnu poziciju u korjenskom direktorijumu fajl sistema i smješten u

¹ Program za pokretanje operativnog sistema

² Programi za pokretanje operativnog sistema – prve faze

³ Volume Boot Record – vrsta sektora za pokretanje koju je uveo IBM

vidu uzastopnih sektora. Boot loader je onda mogao učitati prva tri sektora fajla u memoriju, koji su sadržavali drugi ugrađeni boot loader koji je mogao da učita ostatak fajla.

Dodavanjem LBA (*Logical Block Addressing*) i FAT32 podrške, Microsoft je prešao na boot loader koji je dostigao preko 2 fizička sektora i koristio 386 instrukcija radi veličine. Istovremeno, drugi proizvođači su uspjeli integrisati mnogo više funkcionalnosti u jedan sektor bez redukovanja originalnih ograničenja vezanih za raspoloživu memoriju (32KB) i procesorsku podršku (8088/8086). Kao primjer možemo navesti DR-DOS sektore za pokretanje koji su mogli locirati i učitati u cjelini fajl za pokretanje na FAT12, FAT16 i FAT32 sistemima, čak i ako fajl nije smješten na fiksnu lokaciju i u uzastopnim sektorima.

Primjeri FSBL su coreboot, Libreboot i Das U-Boot.

Boot loader druge faze (*Second-stage boot loader*)

Boot loaderi druge faze, kao što su GNU GRUB, rEFInd, BOOTMGR, Syslinux, NTLDR ili BootX, sami po sebi nisu operativni sistemi, ali jesu u mogućnosti da učitaju ispravno operativni sistem i predaju mu izvršavanje. Operativni sistem se naknadno inicijalizuje i može učitati dodatne drajvere. Boot loader druge faze ne treba drajvere da bi radio, ali umjesto njih može koristiti generičke metode za pristup skladištu koje obezbjeđuje sistemski firmware kao što je BIOS ili OpenFirmware, ali sa ograničenom hardverskom funkcionalnosti i lošijim performansama. Mnogi programi za pokretanje (kao npr. GNU GRUB, BOOTMGR, NTLDR) mogu biti konfigurisani tako da daju korisniku nekoliko izbora za pokretanje. Ovi izbori mogu da uključuju različite operativne sisteme (za dual i multi-boot), različite verzije istog operativnog sistema, različite režime za učitavanje operativnog sistema (npr. safe mode) i neke samostalne programe koji mogu da funkcionišu bez OS (npr. memtest86+, osnovni shell ili čak i neke igrice). Neki boot loaderi mogu da učitaju druge boot loadere, npr. GRUB učitava BOOTMGR umjesto direktno Windows. Obično je podrazumijevani izbor selektovan istekom nekog vremenskog intervala unutar kojeg korisnik može pritiskom nekog tastera taj izbor da promijeni. Ukoliko se to ne desi, pokretanje se vrši bez interakcije, podrazumijevano. Proces pokretanja se smatra završenim onda kada je računar spreman da interaguje sa korisnikom ili je OS u mogućnosti da pokreće sistemske programe i aplikacije.

Mnogi ugrađeni sistemi moraju da se pokrenu trenutno, npr. čekanje minut da se pokrene TV ili GPS uređaj smatra se neprihvatljivim. Radi toga, takvi uređaji imaju softverske sisteme u ROM ili fleš memoriji tako da uređaj može u trenutku početi funkcionisati – potrebno je malo ili nimalo učitavanja.

Veliki i složeni sistemi mogu da imaju procedure za pokretanje u nekoliko faza dok se operativni sistem i drugi programi ne učitaju i osposobe za izvršavanje. S obzirom na to da su operativni sistemi dizajnirani tako da nemaju početak ni kraj, program za pokretanje može da učita operativni sistem, konfigurise se kao najobičniji proces unutar tog sistema, nepovratno preda kontrolu operativnom sistemu, a zatim terminira kao što bi svaki drugi proces.

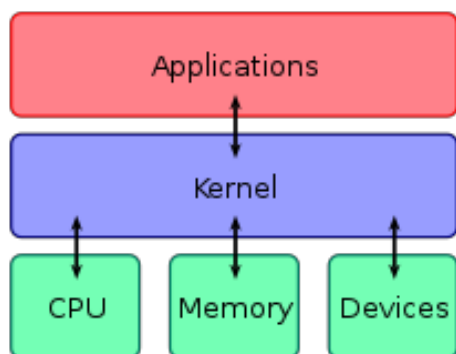
Mrežno pokretanje

Mnogi računari su u mogućnosti da izvrše boot proces preko računarske mreže. U ovom slučaju, operativni sistem je smješten na disku servera i dijelovi njega se prenose do klijenta upotrebom jednostavnog protokola kao što je TFTP (Trivial File Transfer Protocol) Nakon što su ti dijelovi preneseni, OS preuzima kontrolu nad procesom pokretanja. Kao i u slučaju sa programima za pokretanje druge faze, mrežno pokretanje počinje upotrebom generičkih metoda

za mrežni pristup koji obezbeđuje ROM. Pomenute metode koristi za pokretanje mrežnog interfejsa koji uglavnom sadrži PXE (Preboot Execution Environment) sliku. Dakle, nisu potrebni nikakvi drajveri, ali je funkcionalnost sistema ograničena dok ne počnu s radom kernel OS-a i drajveri. Kao rezultat, jednom kada je pokretanje zasnovano na ROM-u završeno, moguće je izvršiti mrežno pokretanje operativnog sistema koji sam po sebi nema mogućnost da koristi mrežni interfejs.

2. Kernel i tipovi kernela ^[2]

Kernel je računarski program koji je samo jezgro operativnog sistema, sa potpunom kontrolom nad svime u sistemu. Predstavlja jedan od prvih programa koji se učitava prilikom pokretanja (nakon bootladera). Kernel upravlja ostatkom pokretanja, kao i ulaznim/izlaznim zahtjevima iz softvera, pretvarajući ih u uputstva za obradu podataka za CPU. On upravlja memorijom i perifernim uređajima kao što su tastatura, monitor, štampač itd.

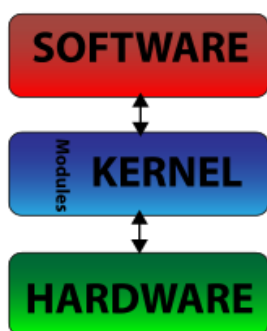


Kritični kod kernela se obično učitava u zasebnu oblast memorije koja je zaštićena od pristupa aplikativnog softvera ili drugih manje kritičnih dijelova operativnog sistema. Kernel izvršava svoje zadatke, kao što su rad sa procesima, upravljanje hardverskim uređajima (HDD, mrežni adapteri...) i rukovanje prekidima u zaštićenom prostoru. Shodno tome sve što korisnik radi se izvršava u korisničkom prostoru. Ovo razdvajanje sprječava miješanje podataka korisnika i kernela, te sprječava neispravne ili maliciozne aplikativne programa da ruše čitav operativni sistem. Kada proces zahtijeva uslugu

kernela, to nazivamo sistemskim pozivom. Primarna funkcija kernela je posredovanje između korisnika i računarskih resursa, odnosno upravljanje CPU-om, memorijom i uređajima.

Tipovi kernela:

Monolitni kernel



Monolitni kernel je tip arhitekture kernela u kojoj cijeli operativni sistem radi u prostoru kernela (isti adresni prostor, veća brzina). Neki proizvođači kvalifikuju monolitni kernel kao jednostavniji za implementaciju nego mikrokernela. Glavni nedostatak monolitnih kernela je zavisnost između sistemskih komponenti – bug u drajveru uređaja može da dovede do rušenja čitavog sistema. Drugi nedostatak je to što je veoma teško održavati velike kernele.

Monolitni kernel, koji je tradicionalno korišten u Unix i srodnim sistemima, sadrži sve osnovne funkcije operativnog sistema i drajvere uređaja. Monolitni kernel je jedan program koji sadrži sav kod neophodan da se izvrši svaki task u vezi sa kernelom. Svaki dio kojem bi trebalo da pristupa većina programa koji ne mogu da se smjeste u biblioteku nalazi se u prostoru kernela: drajveri uređaja, raspoređivač, upravljanje memorijom, fajl sistemi i mrežni stekovi. Aplikacijama se stavljaju na raspolaganje mnogi sistemski pozivi koji im omogućavaju pristup svim ovim uslugama. Moderni monolitni kerneli, kao oni kod Linux-a i FreeBSD-a (oba spadaju u Unix slične OS), imaju mogućnost da učitaju module u runtime-u, čime je omogućeno proširenje mogućnosti kernela onda kada je to potrebno, pri čemu je minimizovana količina koda koji se izvršava u prostoru kernela. Neke prednosti monolitnih kernela su:

- Pošto je manje softvera uključeno, brzina je veća;
- Kako se radi o jednom softveru, on bi trebao da bude manji i u izvornom i kompajliranom obliku;
- Manje koda uopšteno znači manje bug-ova, što povlači manje sigurnosnih problema.

Kao što je već rečeno, većina posla u monolitnim kernelima se obavlja pomoću sistemskih poziva. Sistemski pozivi su najčešće predstavljeni kao interfejsi koji se čuvaju u tabelarnim strukturama i pristupaju nekom podsistemu unutar kernela, kao što su disk operacije. U osnovi, pozivi se vrše unutar programa i validirana kopija zahtjeva se proslijeđuje kroz sistemski poziv. Taj put uopšte nije dug. Monolitni Linux kernel može da bude ekstremno mali ne samo zbog mogućnosti da dinamički učitava module nego i zbog jednostavnosti prilagođavanja. Postoje verzije koje su dovoljno male da zajedno sa velikim brojem uslužnih i drugih programa stanu na jedan floppy disk i svejedno osiguraju potpuno funkcionalan operativni sistem (najpopularniji primjer je muLinux). Ovo je dovelo do široke upotrebe GNU/Linux-a u ugrađenim sistemima.

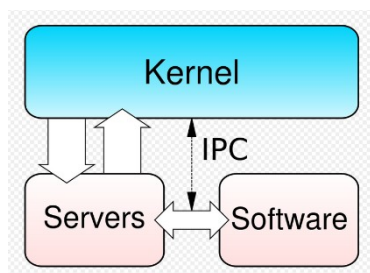
Dakle, ovaj tip kernela se sastoji od osnovnih funkcija OS-a i drajvera uređaja sa mogućnošću učitavanja modula tokom izvršavanja. Obezbeđuju bogate i moćne apstrakcije hardvera. Broj tih apstrakcija je mali, te da bi osigurali više funkcionalnosti koriste se aplikacije koje se nazivaju serveri. Ovaj pristup definiše virtuelni interfejs visokog nivoa nad hardverom sa skupom sistemskih poziva radi implementacije usluga operativnog sistema (upravljanje procesima, konkurentnost i upravljanje memorijom u nekoliko modula koji se pokreću u supervizorskom režimu). Mane i ograničenja ovakvog dizajna:

- Kodiranje u kernelu je izazovno, dijelom zbog toga što je onemogućena upotreba uobičajenih biblioteka, a dijelom zbog toga što je znatno teže efikasno pronalaziti i uklanjati greške u kodu. Onda kada je debugovanje teško, veće su šanse da će kod imati više grešaka;
- Greške u jednom dijelu kernela mogu imati snažan uticaj na ostatak kernela. S obzirom na to da svaka funkcija u kernelu ima sve privilegije, greška u jednoj funkciji može da naruši strukturu podataka druge funkcije ili bilo kog pokrenutog programa;
- Kernel često postane jako velik i težak za održavanje;
- Čak i u slučaju da su neki moduli odvojeni od cjeline, uvezanost koda je i dalje velika;
- Pošto se moduli pokreću u istom adresnom prostoru, jedna greška može da dovede do pada čitavog sistema;
- Nedostatak portabilnosti – za svaku novu arhitekturu na kojoj će se koristiti operativni sistem, mora se sve ispočetka pisati i prilagoditi toj konkretnoj arhitekturi.

Primjeri monolitnih kernela: AIX, HP-UX i Solaris kernel.

Mikrokernel

Mikrokernel je vrsta kernela koja je dizajnirana kao reakcija na tradicionalni monolitni kernel, tako da je funkcionalnost sistema premještena iz tradicionalnog kernela u skup „servera“



koji komuniciraju kroz minimalni kernel, da bi se što manje toga nalazilo u prostoru kernela, a što više u korisničkom prostoru.

Mikrokernel definiše jednostavnu apstrakciju nad hardverom, sa skupom sistemskih poziva za implementaciju minimalnog broja usluga OS-a, kao što su upravljanje memorijom, multitasking i interna komunikacija između procesa. Ostale usluge (one koje je ranije pokrivao kernel – kao npr. umrežavanje) sada su implementirane u programima unutar korisničkog prostora –

serverima. Veličina mikrokernelsa i OS-a je manja i mikrokernel je jednostavniji za održavanje nego monolitni kerneli, ali veliki broj sistemskih poziva i promjena konteksta (context switch) čini sistem sporijim. Jedino dijelovi koji zaista zahtijevaju privilegovani mod se nalaze u prostoru kernela: IPC (Inter-Process Communication), osnovni raspoređivač, osnovno upravljanje memorijom, osnovni I/O primitivi. Mnogi kritični dijelovi su sada u korisničkom prostoru: potpun raspoređivač, upravljanje memorijom, fajl sistemi i mrežni stekovi. Neke prednosti mikrokernelsa:

- Jednostavniji za održavanje, manja veličina.
- Jako kratko vrijeme potrebno za razvoj i proširenje sistema je jednostavnije, novi softver može da se testira bez potrebe za ponovnim pokretanjem kernela (nije bilo moguće kod monolitnih kernela)
- Veća perzistentnost uopšteno.

Većina mikrokernelsa koristi sistem zasnovan na prosljeđivanju poruka kako bi se obrađivali zahtjevi od jednog servera ka drugom. Ovaj sistem funkcionise na bazi portova sa mikrokernelom. Npr. kada se pošalje zahtjev za još memorije, zahtjev se prosljeđuje na port mikrokernelsa. Unutar mikrokernelsa, koraci su slični sistemskom pozivu. Ideja je bila da se modularizuje sistemka arhitektura, što bi učinilo sistem „čistijim“, učinilo debugovanje i dinamičko mijenjanje jednostavnijim i olakšalo prilagođavanje korisnikovim potrebama.

Iako je mikrokernel sam po sebi jako male veličine, u kombinaciji sa svim pomoćnim kodom kojeg koristi, dolazimo do veličine koja nadmašuje veličinu monolitnog kernela. Zagovarači monolitnih kernela ističu da činjenica da OS ne interaguje direktno sa hardverom stvara nezanemarivo manju efikasnost. Naravno, činjenica da kod monolitnih kernela jedna greška ruši čitav sistem (dok je kod mikrokernelsa moguće samo restartovati uslugu koja je uzrokovala grešku, bez uticaja na kernel) služi kao jako dobar kontraargument.

Nedostaci mikrokernelsa:

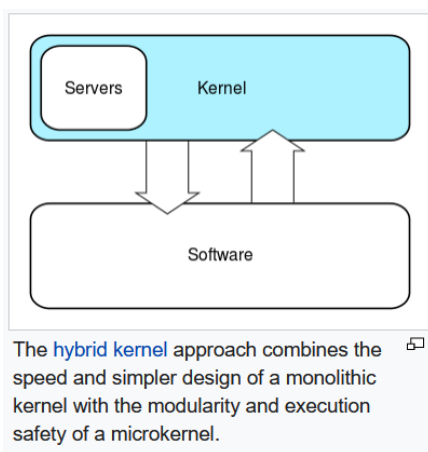
- Veća upotreba RAM-a.
- Potrebno je više softvera za povezivanje, moguća je degradacija performansi.
- Greške prilikom slanja-primanja poruka su teže za otklanjanje s obzirom na to da je put koji treba da pređu duži nego onaj kod monolitnih kernela.
- Upravljanje procesima uopšteno može da bude veoma komplikovano.

Mikrokernel dozvoljava implementaciju dijela operativnog sistema kao uobičajenog programa pisanog u višem programskom jeziku, upotrebu različitih operativnih sistema bez mijenjanja kernela i dinamičko smjenjivanje između operativnih sistema i mogućnost da istovremeno bude aktivno više od jednog OS-a.

Dio su operativnih sistema kao što su GNU Hurd, MINIX, MkLinux, QNX, RedoxOS.

Monolitni kernel	Mikrokernel
Veća veličina, manje koda	Manja veličina, više koda
Bolje performanse, brže izvršavanje jer nema context switching-a između korisničkog i prostora kernela	Lošije performanse, sporije izvršavanje zbog context switching-a između korisničkog i prostora kernela
OS je složen za dizajniranje	Jednostavnije dizajniranje, implementacija i instaliranje OS-a
Sve usluge OS-a su u kernelu	Samo IPC i usluge niskog nivoa su u kernelu
Teže dodati nove funkcionalnosti	Lakše dodati nove funkcionalnosti
Zakazivanje jedne komponente dovodi do zakazivanja sistema i ostalih komponenti	Zakazivanje jedne komponente ne utiče na sistem i ostale komponente

Hibridni (Modularni) kernel



Hibridni kernel je sličan mikrokernelu, osim što uključuje dio dodatnog koda unutar prostora kernela, kako bi se poboljšale performanse. Ova vrsta kernela je zapravo vid kompromisa koji kombinuje glavne prednosti mikrokernela i monolitnog kernela. Za razliku od monolitnih kernela, hibridni kernel nije u mogućnosti da sam učita module tokom izvršavanja. Hibridni kernel možemo shvatiti kao mikrokernel koji ima neki nenužan kod koji bi se inače nalazio u korisničkom prostoru. Tako, neke usluge poput mrežnog steka ili fajl sistema su smještene u prostoru kernela, ali su i dalje prisutni razni serveri u korisničkom prostoru.

Neke prednosti hibridnog kernela:

- Kraće vrijeme razvoja za drajvere koji mogu da funkcionišu unutar modula. Nije potreban reboot za testiranje (rad kernela neće biti narušen)
- Nije potrebno rekompajlirati cijeli kernel zarad dodavanja novih funkcionalnosti, drajvera ili podsistema.
- Prednost u vezi sa prethodnom, ali je vrijedi izdvojiti kao zasebnu – brža integracija third party tehnologija.

Moduli komuniciraju sa kernelom korištenjem neke vrste interfejsa koji bi trebao biti uopšten (u praksi je ipak jedinstven za svaki OS) pa nije uvijek moguće koristiti module. Često drajveri uređaja zahtijevaju više fleksibilnosti nego što pruža interfejs modula.

Neki nedostaci hibridnog kernela:

- Više sigurnosnih rupa, jer se zbog toga što je potrebno proći kroz više interfejsa povećava mogućnost za više grešaka.
- Održavanje modula može biti zbunjujuće.

Hibridni kerneli se koriste u većini komercijalnih operativnih sistema kao što su Microsoft Windows NT 3.1, NT 3.5, NT 3.51, NT 4.0, 2000, XP, Vista, 7, 8, 8.1 i 10. Apple-ov macOS koristi hibridni kernel XNU.

Nanokernel

Nanokernel delegira virtuelno sve usluge (uključujući i najosnovnije poput kontrolera prekida ili tajmera) drajverima uređaja kako bi učinili kernel čak manjim od mikrokernela.

Exokernel

Exokernel je eksperimentalni pristup dizajniranju operativnog sistema. Razlikuje se od ostalih tipova kernela po pitanju funkcionalnosti koja je ograničena na zaštitu i multipleksiranje hardvera, bez pružanja hardverskih apstrakcija. Ovo odvajanje hardverske zaštite od hardverskog upravljanja omogućava projektantima da odrede kako za svaki specifični program obezbijediti najefikasniju upotrebu. Exokernel je ekstremno male veličine, ali ga prati biblioteka operativnih sistema, obezbjeđujući funkcionalnosti konvencionalnog operativnog sistema. Glavna prednost sistema sa exokernelom je što mogu uključiti nekoliko biblioteka operativnih sistema, gdje svaki ima različit API, npr. jedan za razvoj korisničkog interfejsa na visokom nivou, a jedan za kontrolu u realnom vremenu.

3. Prstenovi i privilegije ^[3]

Hijerarhijski zaštitni domeni – zaštitni prstenovi – su mehanizmi zaštite podataka i funkcionalnosti od grešaka (poboljšavanjem otpornosti na greške) i malicioznog ponašanja (obezbjeđujući računarsku zaštitu i sigurnost). Zaštitni prsten predstavlja nivo privilegija unutar arhitekture računara. Prstenovi su raspoređeni hijerarhijski – počevši od najprivilegovanijeg (najviše ovlaštenja, uglavnom numerisan sa 0) do najmanje privilegovanog (najmanje ovlaštenja, numerisan najvećim brojem). U većini OS, prsten 0 ima najviše privilegije i interaguje direktno sa fizičkim hardverom (CPU i memorijom)

Dozvoljeno je da na predefinisani način spoljašnji prstenovi pristupaju resursima unutrašnjih, umjesto da proizvoljno koriste te resurse. Ispravnim definisanjem pristupa moguće



je poboljšati sigurnost sprječavanjem programa iz jednog nivoa privilegija da zlopotrebljavaju resurse namijenjene programima drugog nivoa. Npr. spyware pokrenut kao korisnički program u prstenu 3 ne smije biti u mogućnosti da upali web kameru bez obavještanja korisnika, s obzirom na to da je hardverski pristup drajverima uređaja unutar prstena 1. Web browser koji je pokrenut kao program u prstenu višeg nivoa privilegija mora tražiti pristup mreži, resurs ograničen na prsten nižeg nivoa.

Većina sistema opšte namjene koristi dva prstena, prsten 0 koji odgovara kernel modu i prsten 3 koji odgovara korisničkom modu. Takav sistem je npr. Windows 7. Postoje sistemi koji koriste npr. 3 (OS/2), 4 (OpenVMS) ili 8 (Multics) prstenova. U nekim operativnim sistemima zaštitni prstenovi mogu da se kombinuju sa procesorskim modovima (privilegovani/kernel/master/supervizorski naspram neprivilogovanom/korisničkom/slave). Efektivna upotreba zaštitnih prstenova zahtijeva blisku saradnju hardvera i OS-a. Često je sigurnosni model sveden samo na kernel/korisnik iako prstenovi omogućavaju finije razlike.

Supervizorski režim je režim izvršavanja na nekim procesorima koji omogućava izvršavanje svih instrukcija, uključujući privilegovane. Takođe, može dati pristup drugom adresnom prostoru, hardveru za upravljanje memorijom i drugim periferijama. Ovo je režim u kojem je obično pokrenut OS.

U monolitnom kernelu, OS je pokrenut u supervizorskom režimu, a aplikacije u korisničkom. Ostali tipovi kernela ne prate nužno ovo ponašanje. Linux, macOS i Windows koriste supervizor/korisnički režim. Da bi izvršio određene specijalizovane funkcije, kod korisničkog režima mora da uputi sistemski poziv supervizorskom režimu ili čak prostoru kernela, gdje će se izvršiti zadatak, a zatim vratiti kontrolu korisničkom prostoru. DOS, kao i neki drugi jednostavni operativni sistemi i mnogi ugrađeni uređaji rade trajno u supervizorskom

režimu. x86 procesori imaju 4 različita režima podijeljena u 4 različita prstena. Programi pokrenuti u prstenu 0 mogu da „rade bilo šta sa sistemom“, dok oni koji rade u prstenu 3 treba da mogu u bilo kom trenutku da zakažu bez da utiču na ostatak sistema. Prsten 1 i prsten 2 se rijetko koriste, ali se mogu konfigurisati različitim nivoima pristupa. Noviji procesori Intel-a i AMD-a nude x86 virtuelizacijske instrukcije za hipervizorski nadzor za kontrolu hardverskog pristupa prstenu 0. Oni kreiraju novi prsten – „Ring - 1“ kako bi gostujući operativni sistem mogao da pristupa prstenu 0 bez da utiče na druge gostujuće sisteme ili host sistem. Vršiti se dodavanje novih 9 mašinskih instrukcija koje rade samo na „Ring - 1“, odnosno predviđeno je da ih koristi samo hipervizor.

Nivo privilegija u x86 instrukcionom setu kontroliše pristup programa koji se trenutno izvršava na procesoru resursima kao što su memorija, I/O portovi i specijalne instrukcije. Postoje 4 nivoa privilegija, numerisana od 0 (najprivilegovaniji) do 3 (najmanje privilegovan). Većina modernih operativnih sistema koristi dva nivoa, 0 za kernel i 3 za aplikacije. Svaki resurs koji je na raspolaganju nivou n je takođe na raspolaganju nivoima od 0 do n . Kada manje privilegovan proces pokuša da pristupi resursima privilegovanijeg, GPF (General Protection Fault) prekid se šalje operativnom sistemu. Operativni sistemi Microsoft Windows, macOS, Linux, iOS i Android većinom koriste paging mehanizam sa samo jednim bitom kako bi kvalifikovali nivo privilegija kao supervizorski ili korisnički (S/U bit). Windows NT koristi sistem sa dva nivoa.

U budućnosti, na polju virtuelnih mašina, kernel host OS-a bi mogao koristiti instrukcije najvišeg nivoa (kernel mod), dok bi aplikacije pokrenute na gostujućem OS-u mogle koristiti najniže privilegije (korisnički mod). Virtuelna mašina i gostujući OS bi mogli koristiti neki srednji nivo privilegija kako bi mogli inicirati operacije poput sistemskih poziva na nivou gostujućeg OS-a.

IOPL (I/O Privilege level) je flag koji zauzima 12. i 13. bit u FLAGS registru. Pokazuje I/O nivo privilegija trenutnog programa ili taska. CPL (Current Privilege Level – CPL0, CPL1, CPL2, CPL3) taska ili programa mora biti manji ili jednak IOPL kako bi mogao pristupiti I/O portovima.

4. MBR i GPT ^[4]

Master Boot Record (MBR) predstavlja specijalni tip boot sektora koji se nalazi na samom početku uređaja za skladištenje podataka. MBR je javno predstavljen 1983. sa pojavom PC DOS 2.0 OS-a. MBR sadrži podatke o organizaciji uređaja na kojima se čuvaju fajl sistemi. Pored njih, MBR sadrži izvršni kod koji služi kao loader za inicijalizaciju instaliranog operativnog sistema – obično prosljeđivanjem kontrole izvršavanja loaderu druge faze ili u saradnji sa VBR-om (Volume Boot Record) svake particije. Ovaj kod se obično naziva boot loader.

Organizacija tabele particionisanja kod MBR-a ograničava adresni prostor particionisanog diska na 2 TiB ($2^2 \times 512$ bajtova). Pokušaji da se ovaj limit poveća nisu uspjeli jer nisu bili kompatibilni sa postojećim MBR podržanim OS-ima, sistemskim alatima i boot loaderima te često dovode do ozbiljne korupcije podataka ako se koriste van usko definisanih ograničenja. Posljedica gore navedenog je da šeme particionisanja zasnovane na MBR-u bivaju zamijenjene GPT-om (GUID Partition Table) u novijim računarima. GPT može koegzistirati sa MBR-om u svrhu kompatibilnosti particija unazad.

GPT takođe predstavlja standardan način organizacije tabele particija na uređajima za čuvanje podataka. Koristi univerzalno jedinstvene identifikatore (GUID – Globally Unique Identifier). Čini dio UEFI (Unified Extensible Firmware Interface) standarda koji je predložen kao zamjena za klasični BIOS, ali se koristi i u sistemima koji koriste BIOS gdje 32-bitno MBR adresiranje nije dovoljno.

Svi moderni računari podržavaju GPT. Pojedini OS, kao što su macOS i Microsoft Windows podržavaju pokretanje sistema sa GPT particija samo u sistemima koji imaju EFI firmware dok FreeBSD i većina Linux distribucija mogu biti pokrenute sa GPT particija bez obzira na interfejs koji se koristi (BIOS, EFI).

5. Procesi i model procesa ^[5]

Centralni koncept svakog OS-a je proces - apstrakcija pokrenutog programa i jedna od najstarijih i najvažnijih apstrakcija vezanih za OS. Procesi se konkurentno izvršavaju na jednom procesoru, pretvarajući taj procesor u više virtuelnih procesora. Kada se sistem pokrene, mnogi procesi započnu bez znanja korisnika. Istovremeno, eksplicitni korisnički procesi se takođe počnu izvršavati. Multiprogramiranje podržava upravljanje sa više procesa. U svakom sistemu koji podržava multiprogramiranje, CPU brzo prelazi sa procesa na proces, izvršavajući svaki desetak ili hiljade milisekundi. To daje iluziju paralelizma. Ovo se češće naziva pseudoparalelizam kako bi se razlikovao od stvarnog hardverskog paralelizma koji obezbjeđuju multiprocesorski sistemi.

U modelu procesa, svaki softver na računaru, ponekad uključujući operativni sistem, organizovan je u veliki broj sekvencijalnih procesa. Proces je zapravo samo instanca programa koji se izvršava, uključujući trenutne vrijednosti PC (Program Counter) registra i ostalih registara i varijabli. Konceptualno, svaki proces ima svoj vlastiti virtuelni CPU. U stvarnosti, jedan CPU prelazi sa procesa na proces izvršavajući djelić svakog (multiprogramiranje). Radi lakšeg razumijevanja, jednostavnije je zamisliti kolekciju procesa koji se izvršavaju pseudoparalelno.

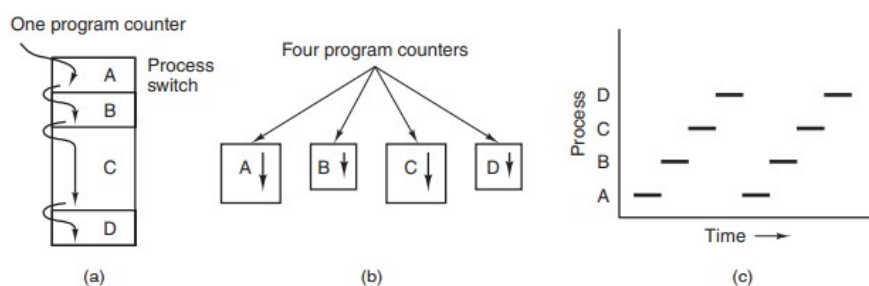


Figure 2-1. (a) Multiprogramming four programs. (b) Conceptual model of four independent, sequential processes. (c) Only one program is active at once.

Na slici (a) vidimo multiprogramiranje četiri procesa u memoriji, na slici (b) vidimo 4 procesa, svaki sa svojom kontrolom toka (vlastitim logičkim PC) i svaki se izvršava

nezavisno od ostalih. Naravno, postoji samo jedan stvarni PC, pa svaki put kada se neki proces izvršava, njegov logički PC se upisuje u stvarni PC. Kada se trenutno izvršavanje završi, fizički PC se smješta u logički PC. Na slici (c) vidimo da su svi procesi napredovali u izvršavanju, ali se u svakom trenutku izvršava samo jedan proces.

Pretpostavimo sada da postoji samo jedan CPU. Rekosmo da taj CPU može da izvršava samo jedan proces u trenutku, pa 2 procesora mogu da izvršavaju 2 procesa istovremeno (multiprocesorski sistemi). Različiti procesi se izvršavaju sa različitim vremenima izračunavanja, pa se procesi ne smiju programirati sa ugrađenim pretpostavkama o tajmingu. Npr. zamislimo audio proces koji reprodukuje muziku u pratnji sa HQ video sadržajem koji je pokrenut na drugom uređaju. Pošto audio treba da započne malo kasnije nego video, šalje se signal video serveru da započne reprodukciju video sadržaja, a onda započinje praznu petlju 10000 puta prije nego što počne audio. To funkcioniše dobro ako je petlja pouzdan tajmer, ali ukoliko CPU odluči da pređe na drugi proces tokom petlje, doći će do neželjenog ponašanja. Kada proces ima kritične zahtjeve u realnom vremenu (poput ovog), određeni događaji moraju

da se dese unutar specifikovanog broja milisekundi i posebne mjere treba da se poduzmu da bi se osiguralo da će se to desiti. Inače, na većinu procesa zapravo ne utiče pozadinsko multiprogramiranje ili brzina izvršavanja drugih procesa.

Razlika između procesa i programa je suptilna, ali izuzetno značajna. Slikovit primjer: zamislimo kulinarski orijentisanog programera koji pravi tortu za rođendan svoje kćerke. On ima recept i kuhinju sa svim sastojcima: brašno, jaja, šećer, itd. U ovom primjeru, recept je program – algoritam izražen prikladnom notacijom, programer je procesor (CPU), a sastojci su ulazni podaci. Proces je aktivnost koja se sastoji od programerovog čitanja recepta, dohvaćanja sastojaka i pravljenja torte. Zamislimo da ulijeće programerov sin koji kaže da ga je ubola pčela. Programer upamti gdje je stao sa receptom (pamti stanje trenutnog procesa), uzima knjigu iz prve pomoći i počinje pratiti uputstva iz nje. Tako vidimo prelazak sa jednog procesa na proces višeg prioriteta, pri čemu svaki ima drugi program (recept i knjiga). Nakon što pomogne sinu, programer nastavlja sa pravljenjem torte od mjesta na kome je prekinut.

Ključna ideja ovdje je da je proces zapravo aktivnost. On ima svoj program, ulaze, izlaze i stanje. Jedan procesor dijeli više procesa, sa nekim algoritmom za raspoređivanje kako bi bilo jasno kada prestati sa jednim procesom i uslužiti drugi. Nasuprot tome, program je nešto što se nalazi na disku, a ne nekakva radnja. Ukoliko se isti program izvršava dva puta, predstavlja dva različita procesa. Moguće je da operativni sistem dijeli kôd između njih, tako da postoji samo jedna kopija u memoriji, ali to ne mijenja činjenicu da se izvršavaju dva procesa.

6. Kreiranje i terminiranje procesa ^[6]

Operativni sistem mora da ima neki način da kreira procese. U jednostavnim sistemima ili sistemima koji su namijenjeni samo za jednu svrhu (npr. kontroler u mikrovalnoj), moguće je da imamo sve procese koji će se ikad kreirati onda kada se sistem pokrene. U sistemima opšte namjene, neophodno je imati način za kreiranje i terminiranje procesa u skladu s potrebama tokom izvršavanja. Četiri osnovna događaja uzrokuju kreiranje procesa: sistemska inicijalizacija, izvršavanje sistemskog poziva za kreiranje procesa od strane procesa koji se trenutno izvršava, korisnički zahtjev za kreiranjem procesa i iniciranje batch posla⁴. Nakon što je operativni sistem pokrenut, mnoštvo procesa je kreirano. Neki od njih su foreground (površinski) procesi – procesi koji interaguju sa korisnicima i obavljaju određen posao za njih. Drugi su background (pozadinski) i nisu asocirani sa konkretnim korisnicima nego imaju neku specifičnu funkciju. Pozadinski procesi namijenjeni za prihvatanje zahtjeva za web stranicama, email-om, printanjem i sl. nazivaju se daemoni⁵. U Unix-u, *ps* program može da se koristi da bi se vidjeli pokrenuti procesi, u Windows-u koristi se task manager. Osim procesa pokrenutih prilikom pokretanja OS-a, novi procesi pristižu nakon toga. Obično pokrenuti proces šalje sistemske pozive za kreiranje novih procesa kako bi mu pomogli da se izvrši. Kreiranje novih procesa je takođe korisno kada imamo procese koji su povezani, ali inače odvojeni procesi koji interaguju. Npr. ako je potrebno pribaviti veliku količinu podataka sa mreže za naknadnu obradu, moglo bi biti korisno napraviti jedan proces za dohvaćanje podataka i smještanje u dijeljeni bafer, dok bi drugi proces uklanjao podatke odatle i obrađivao ih. U interaktivnim sistemima, korisnik može da pokrene program unosom komande ili dvostrukim klikom na neku ikonu. Obe ove radnje započinju novi proces sa datim programom. U UNIX sistemima zasnovanim na komandama, novi proces preuzima prozor u kojem je pokrenut. U Windows-u, proces nema svoj prozor osim ako ga ne kreira (uglavnom se to i dešava). U oba sistema, korisnici mogu da imaju više prozora otvorenih istovremeno takvih da svaki pripada nekom procesu. Korisnik upotrebom periferija može da interaguje sa procesom, npr. obezbjeđujući potreban ulaz. U UNIX-u postoji samo jedan sistemski poziv za kreiranje novog procesa: *fork*. Ovaj poziv kreira klon pozivajućeg procesa. Nakon toga, dva procesa, roditeljski i dijete imaju istu memorijsku sliku, iste stringove okruženja i iste otvorene fajlove. Obično, proces dijete poziva *execve* ili neki drugi sistemski poziv da bi promijenio memorijsku sliku i pokrenuo novi program. U Windowsu, jedan Win32 sistemski poziv – *CreateProcess* obrađuje i kreaciju procesa i učitavanje ispravnog programa u novi proces. Pored ovog sistemskog poziva, Win32 ima oko 100 drugih funkcija za upravljanje i sinhronizaciju procesa i srodne poslove. I u UNIX i u Windows sistemima, nakon što je proces kreiran, roditeljski i dijete proces imaju svoj vlastiti adresni prostor. Ako jedan proces promijeni nešto u svom adresnom prostoru, promjena se ne odražava na drugi proces. U UNIX-u, inicijalni adresni prostor djeteta je kopija roditeljskog, ali su to dva različita adresna prostora. Neke UNIX implementacije dijele tekst programa jer se on ne mijenja. Alternativno, dijete može da dijeli memoriju sa roditeljskim procesom, ali u tom slučaju ukoliko jedan proces traži promjenu dijela memorije, taj dio memorije se eksplicitno

⁴ <https://www.bmc.com/blogs/batch-jobs/>

⁵ [https://en.wikipedia.org/wiki/Daemon_\(computing\)](https://en.wikipedia.org/wiki/Daemon_(computing))

kopira kako bi se osiguralo da je to privatni memorijski prostor. Moguće je takođe da dijete proces dijeli resurse sa roditeljskim (npr. otvorene fajlove). U Windows-u, prostor roditeljskog i procesa djeteta su od starta drugačiji.

Nakon što je proces kreiran i obavi posao, dolazi do terminiranja. Proces će terminirati uglavnom iz sljedećih razloga: normalan završetak (dobrovoljan), završetak s greškom (dobrovoljan), fatalna greška (obavezan), terminirao ga drugi proces (obavezan). Prvi razlog - Većina procesa ima normalan završetak. Postoji sistemski poziv za završetak – *exit* ili ikonica koju korisnik može da klikne kako bi proces raspustio privremene fajlove koje drži otvorene i onda terminirao. Drugi razlog - ukoliko proces otkrije da postoji greška, može da terminira, npr. ukoliko korisnik unese komandu `cc foo.c` da kompajlira program `foo.c` i takav program ne postoji, kompajler jednostavno saopšti grešku i terminira. Neki programi (screen-oriented) ne terminiraju kada se unesu neispravni argumenti, nego uglavnom traže od korisnika da pokuša ponovo. Treći razlog – uglavnom programski bug, npr. problem sa referenciranjem memorije, dijeljenje sa nulom.. U nekim sistemima (UNIX) proces može da saopšti OS-u da on želi da obradi grešku sam i u tom slučaju proces prima signal (interrupt) umjesto da bude terminiran. Četvrti razlog – neki proces traži od OS da terminira neke druge procese. U UNIX-u ovaj sistemski poziv je *kill*. U Windows-u to je *TerminateProcess*. U oba slučaja, proces koji pruža zahtjev mora da ima ovlaštenja za terminiranje drugih procesa. U nekim sistemima, kada je proces terminiran iz bilo kojeg razloga, svi procesi koje je on kreirao su takođe terminirani. To nije slučaj sa UNIX-om i Windows-om.

7. Stanja procesa ^[7]

Iako svaki proces predstavlja nezavisan entitet, sa svojim vlastitim PC-om i internim stanjem, procesi često treba da interaguju sa drugim procesima. Jedan proces može da generiše izlaz koji je ulaz drugom procesu. Kada proces blokira, to se dešava jer logički ne može nastaviti dalje, obično jer čeka na ulaz koji još nije raspoloživ. Moguće je i da proces bude spreman za izvršavanje ali čeka jer je OS odlučio da dodijeli CPU drugom procesu. U prvom slučaju suspendovanje je svojstveno problem (ne može se procesirati ulaz koji još nije unesen), a u drugom slučaju pitanje je tehničko (nema dovoljno CPU-a da se dodijeli svakom procesu). Na slici vidimo dijagram koji prikazuje proces u tri stanja u kojim može da se nađe: running (izvršava se, koristi CPU), ready (spreman da se izvršava, ali privremeno čeka da se drugi proces izvrši), blocked (ne može da se izvršava dok se ne desi neki spoljašnji događaj). Prva dva stanja su slična, proces bi se u oba izvršavao, ali u drugom nema CPU raspoloživ. Treće stanje je fundamentalno drugačije. Za ova tri stanja, moguća su 4 prelaza. Prelaz 1 se dešava kada OS otkrije da proces sada ne može da nastavi. U nekim sistemima proces može da izvrši sistemski poziv (npr. *pause*) da bi došao u stanje blocked. U drugim sistemima, kada proces čita ulaz a ulaz nije dostupan, automatski biva blokiran. Prelazi 2 i 3 su uzrokovani raspoređivačem procesa, bez da proces uopšte zna za to. Prelaz 2 se dešava kada raspoređivač ustanovi da se proces izvršavao dovoljno dugo i da je vrijeme da se izvršava neki drugi proces. Prelaz 3 se javlja kada su svi procesi imali dio vremena za izvršavanje i vrijeme je da se krene ispočetka, tj. da prvi proces dobije svoje vrijeme ponovo. Prelaz 4 se javlja kada se desi spoljašnji događaj na koji je proces čekao (npr. unesen ulaz). Ako ne postoji drugi proces koji se trenutno izvršava, desiće se prelaz 3 i proces će započeti izvršavanje. U suprotnom će neko vrijeme da se nalazi u stanju ready. Neki procesi koriste komande koje unosi korisnik, neki su dio sistema. Umjesto da razmatramo prekide, možemo da razmatramo korisničke procese, procese diska, procese terminala itd. koji blokiraju kada čekaju da se nešto desi.

SEC. 2.1

PROCESSES

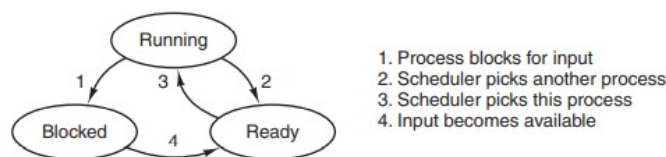


Figure 2-2. A process can be in running, blocked, or ready state. Transitions between these states are as shown.

8. Niti i upotreba niti ^[8]

U tradicionalnim operativnim sistemima svaki proces ima adresni prostor i kontrolnu nit (thread). Uglavnom je poželjno imati više kontrolnih niti u istom adresnom prostoru koje se izvršavaju u kvazi-paraleli kao da su odvojeni procesi – izuzev što dijele adresni prostor. Glavni razlog zbog kojeg bismo htjeli imati ove miniprocese – niti jeste činjenica da se u mnogim aplikacijama više aktivnosti dešava odjednom. Ponekad neke od tih aktivnosti mogu da blokiraju. Ukoliko se takva aplikacija podijeli u više sekvencijanih threadova koji se izvršavaju kvazi-paralelno, programski model se pojednostavljuje. Pored prethodno pomenutih paralelnih procesa, s threadovima dodajemo novi element, mogućnost da paralelni entiteti dijele adresni prostor i sve podatke. Drugi razlog leži u činjenici da su threadovi “lakši” od procesa, jednostavniji su (brži) za kreiranje i uništavanje nego procesi (10 do 100 puta brže). Treći razlog je takođe u vezi sa performansama. Threadovi ne donose bolje performanse kada su svi CPU vezani, ali kada postoji značajno izračunavanje i značajan I/O, postojanje threadova omogućava preklapanje ovih aktivnosti, što ubrzava rad. Takođe, threadovi su korisni na multiprocesorskim sistemima, gdje je stvarni paralelizam moguć. Konkretni primjer je procesor teksta. Oni uglavnom prikazuju dokument na ekranu formatiran tačno onako kako bi bio na printanoj stranici. Elementi se nalaze na svojim finalnim pozicijama. Pretpostavimo da korisnik piše knjigu. Iz tačke gledišta autora, najlakše je čuvati cijelu knjigu kao jedan fajl, kako bi globalne zamjene bile jednostavnije. Alternativno moguće je čuvati poglavlja kao zasebne fajlove. U svakom slučaju, takvo posjedovanje mnoštva fajlova je otežavajuća okolnost kada se rade globalne promjene jer se stotine fajlova moraju individualno editovati, jedan po jedan. Ukoliko je cijela knjiga jedan fajl, obično jedna komanda može da obavi sve zamjene. Ali, ukoliko korisnik odluči da obriše jednu rečenicu sa prve strane knjige koja ima 800 strana i poželi da napravi promjenu na strani 600, tekstualni procesor je prisiljen da reformatira cijelu knjigu do 600. strane jer ne može da zna na koji položaj treba da se prebaci dok ne procesira sve prethodne strane. Stoga, korisnik treba da čeka. Ovdje mogu pomoći threadovi. Zamislimo da je tekstualni procesor program sa 2 threada. Jedan thread interaguje sa korisnikom, a drugi obavlja reformatiranje fajla u pozadini. Čim se obriše rečenica, drugi thread vrši reformatiranje, a prvi “sluša” korisnika i reformatiranje će vjerovatno biti završeno prije nego što korisnik uopšte traži prebacivanje na stranu 600 te će ona biti prikazana trenutno. Što da ne dodamo i treći thread? Npr. automatsko čuvanje fajla svakih nekoliko minuta, da bismo osigurali da korisnik ne izgubi sve podatke usljed neke greške. Kad bi tekst procesor bio sa jednom niti, svaki put kad je bekap pokrenut, ulazi sa tastature i miša se ignorišu dok se bekap ne završi. Postojala bi mogućnost da ulazi predstavljaju prekid, ali bi to dovelo do složenog programskog modela zasnovanog na prekidima. Sa tri threada, model je mnogo jednostavniji. Postojanje 3 procesa ne bi bilo funkcionalno, jer sva tri threada treba da rade na dokumentu. Kada imamo tri threada unutar jednog procesa, svi dijele memoriju i imaju pristup editovanom dokumentu. Threadovi ostvaruju ideju sekvencijalnih procesa koji obavljaju blokirajuće pozive (npr. za I/O uređaj) i ostvarivanje paralelizma uprkos tome. Blokirajući sistemski pozivi olakšavaju programiranje, a paralelizam poboljšava performanse. Threadovi su korisni u aplikacijama koje obrađuju veoma velike količine podataka. Klasičan pristup bi bilo čitanje podataka u bloku, obrada i ponovno upisivanje. Threadovi nude olakšanje – proces treba biti podijeljen u tri threada, ulazni thread, thread za obradu i izlazni thread. Ulazni upisuje podatke u bafer, thread za obradu čita

odatelje i obrađuje i rezultat smješta u izlazni bafer, odakle podatke čita nazad na disk izlazni thread. Ovako se sve tri operacije mogu izvršavati istovremeno.

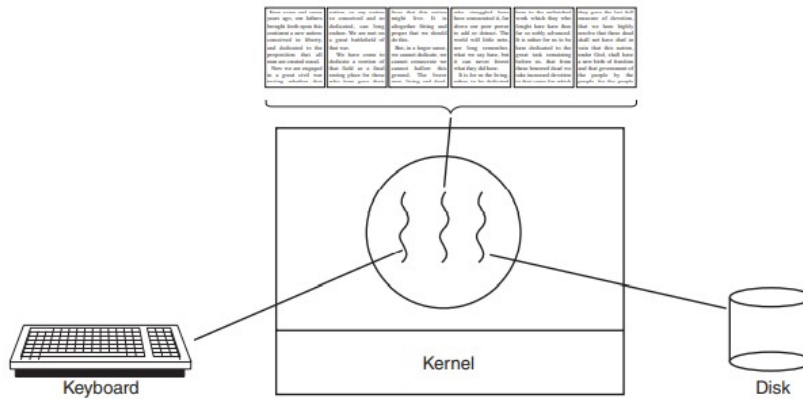


Figure 2-7. A word processor with three threads.

9. Niti - Klasični model ^[9]

Procesni model je zasnovan na dva odvojena koncepta: grupisanje resursa i paralelno izvršavanje. Jedan način gledanja na proces je da je to vid grupisanja povezanih resursa. Ovi resursi mogu da budu otvoreni fajlovi, procesi djeca, alarmi na čekanju.. Grupisanjem u proces, ovi resursi su jednostavniji za upravljanje. Drugi koncept procesa je da je to thread. Thread ima PC koji čuva informaciju o tome koja instrukcija treba da bude izvršena sljedeća. Ima registre, koji čuvaju trenutne varijable. Ima stek, koji čuva istoriju izvršavanja sa jednim frejmom za svaku proceduru koja je pozvana, ali koja nije još vratila rezultat. Iako se thread mora izvršavati u procesu, thread i proces su različiti koncepti i treba da se posmatraju kao takvi. Procesi služe za grupisanje resursa, threadovi su entiteti raspoređeni za izvršavanje na CPU. Ono što threadovi dodaju procesnom modelu je mogućnost da se više niti izvršavanja dešava u istom procesnom okruženju, sa velikom međusobnom nezavisnošću.

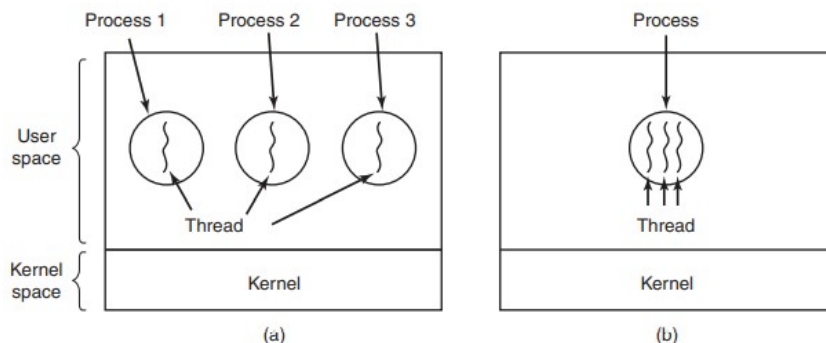


Figure 2-11. (a) Three processes each with one thread. (b) One process with three threads.

Postojanje više threadova u jednom procesu je analogno postojanju više procesa koji se izvršavaju paralelno na jednom računaru. Pošto threadovi imaju neke karakteristike procesa, ponekad se nazivaju “lightweight” procesi. Multithreading je termin koji opisuje

postojanje više threadova unutar jednog procesa. Multithreading funkcioniše kao multiprogramiranje – onako kako procesor prebacuje izvršavanje između procesa kod multiprogramiranja, kod multithreadinga prebacuje izvršavanja sa niti na nit. Različiti threadovi unutar procesa nisu nezavisni onoliko koliko su nezavisni različiti procesi. Pošto dijele isti adresni prostor, dijele i iste globalne varijable. Ne postoji zaštita između threadova, jedan thread može da mijenja stek drugog. Threadovi unutar istog procesa ne dijele samo adresni prostor, nego i otvorene fajlove, procese djecu, alarme, signale...

Per-process items	Per-thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

Figure 2-12. The first column lists some items shared by all threads in a process. The second one lists some items private to each thread.

Kada bi svaki thread imao svoj adresni prostor, otvorene fajlove i ostalo navedeno u koloni 1, to bi bio odvojen proces. Dakle, ono što nastojimo postići sa konceptom threadova je mogućnost da više threadova izvršavanja dijele skup resursa tako da mogu raditi u sprezi da bi obavili zadatak. Kao klasičan proces, thread može da se nalazi u stanjima running, blocked,

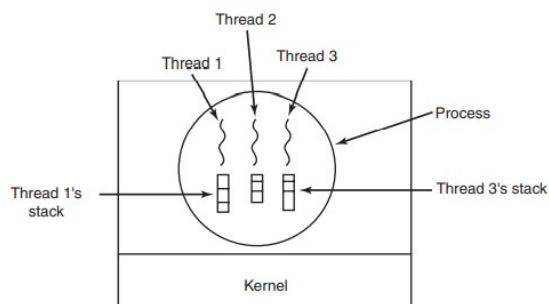


Figure 2-13. Each thread has its own stack.

ready, terminated. Running thread trenutno ima CPU i aktivan je. Blocked thread čeka na neki događaj da ga odblokira. Ready thread će se početi izvršavati čim mu se ukaže prilika. Prelazi između stanja takođe su istovjetni onim kod procesa. Proces inicijalno ima jedan thread, ali taj thread može da kreira nove threadove pozivom bibliotečkih procedura kao što je *thread create*. Ponekad postoji hijerarhijska struktura među threadovima, ali obično se svi posmatraju

jednakim. Kada thread završi posao, može da se završi pozivom bibliotečke procedure *thread exit*. Tada nestaje i više nije raspoloživ za raspoređivanje. U nekim sistemima, jedan thread može da pričekava specifični thread da završi pozivom procedure *thread join*. Taj poziv blokira pozivajući thread dok se taj drugi ne okonča.

Uzimajući sve navedeno u obzir, kreiranje i terminiranje threadova je nalik na isto kod procesa, sa faktički istim mogućnostima na raspolaganju. Još jedan od čestih poziva je *thread yield*, koji omogućava threadu da prepusti svojevoljno CPU da bi se drugi thread izvršavao. Takav poziv je značajan jer ne postoji clock interrupt kakav postoji kod procesa. Poželjno je da threadovi budu u tom smislu “pristojni” i znaju da treba da puste i druge da se izvršavaju s vremena na vrijeme. Threadovi su izuzetno korisni, ali mogu da uvedu neke komplikacije u model. Npr. šta se dešava kod sistemskog poziva *fork*? Da li dijete proces ima onoliko threadova koliko ima roditelj? Ukoliko nema, možda funkcionisanje neće biti ispravno. Ukoliko ima, šta ako je thread kod roditelja blokiran jer čeka na ulaz, da li je blokiran i kod djeteta? Ako dođe ulaz, da li ga dobija samo roditelj ili samo dijete ili oboje? Isti problem postoji sa otvorenim mrežnim konekcijama. Drugi problem je dijeljenje velikog broja struktura podataka. Šta se desi ako jedan thread zatvori fajl dok drugi čita iz njega? Šta ako dva threada istovremeno primijete nedostatak memorije pa je oba alociraju? Ovi problemi mogu da se riješe uz određen napor, ali su pažljivo promišljanje i dizajn potrebni da bi multithreaded programi funkcionisali ispravno.

10. User i kernel-space niti, hibridne i pop-up niti ^[10]

Dva su glavna prostora za implementaciju threadova: korisnički prostor i prostor kernela. Moguća je i hibridna implementacija.

Prvi pristup je implementacija svega vezano za threadove u korisnički prostor. Kernel stoga ne zna ništa o nitima, za njega su svi procesi procesi sa jednim threadom. Očigledna prednost je mogućnost implementacije user-level paketa threadova na operativnom sistemu koji ne podržava threadove. Sa ovim pristupom, threadovi su implementirani bibliotekom.

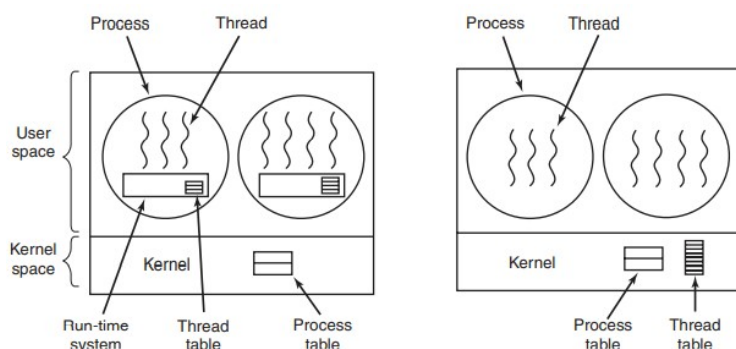


Figure 2-16. (a) A user-level threads package. (b) A threads package managed by the kernel.

Threadovi se izvršavaju na runtime sistemu, koji je kolekcija procedura za upravljanje threadovima. Već smo upoznati sa 4: `pthread_create`, `pthread_exit`, `pthread_join`, `pthread_yield`, ali postoji još mnogo drugih. Kada su threadovi u korisničkom prostoru, svaki proces mora da posjeduje svoju vlastitu thread tabelu. Kada su u pitanju prednosti ovog

pristupa, raspoređivanje threadova je dosta brže, svaki proces može da ima svoj vlastiti algoritam raspoređivanja, threadovi se skaliraju jednostavnije jer threadovi u prostoru kernela ipak zauzimaju određen prostor za tabelu i stek, što može biti problem kad je veliki broj threadova. Uprkos svim navedenim prednostima, postoje značajni nedostaci. Prvi problem je način implementacije blokirajućih sistemskih poziva. Thread može da obavlja blokirajuće sistemske pozive i time stopira sve ostale threadove. Takvi pozivi se mogu učiniti neblokirajućim, ali su neophodne promjene operativnog sistema i korisničkih programa. Alternativa je moguća u slučaju kad unaprijed možemo predvidjeti da li će kod blokirati, što zahtijeva izmjenu dijelova biblioteke sa sistemskim pozivima i neefikasno je i neelegantno, ali nema se puno izbora. Kod smješten oko sistemskog poziva da bi se izvršila provjera naziva se wrapper. Analogno ovom problemu je problem page fault-ova. Ne mora cijeli program odjednom biti u glavnoj memoriji. Ako program skoči na instrukciju koja nije u memoriji, javlja se page fault. Ako thread dovede do page faulta, kernel – nesvjestan postojanja threadova – prirodno blokira cijeli proces, iako bi se ostali threadovi možda mogli izvršavati. Još jedan problem je što kada thread počne da se izvršava, nijedan drugi neće moći da se izvršava dok ovaj svojevrijem ne preda CPU, jer nema clock interrupt-ova. Na kraju, glavna mana je to što programeri žele threadove u aplikacijama gdje threadovi često blokiraju, kao što je višenitni web server. Ovi threadovi konstantno prave sistemske pozive. Ovo eliminiše faktički smisao upotrebe threadova u korisničkom prostoru.

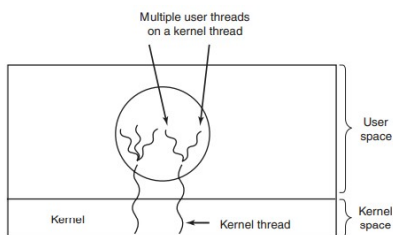


Figure 2-17. Multiplexing user-level threads onto kernel-level threads.

Drugi pristup je da kernel zna za threadove i upravlja njima. Nije potreban run-time sistem za svaki proces, kao ni thread tabela u svakom procesu. Umjesto toga, kernel ima jednu thread tabelu u kojoj vodi evidenciju o svim threadovima u sistemu. Kada jedan thread želi kreiranje ili terminiranje drugog, pravi poziv kernelu koji onda obavlja taj posao ažurirajući thread tabelu. Tabela sadrži sve informacije vezano za svaki thread. Dodatno, kernel takođe čuva i održava tradicionalnu tabelu procesa. Svi pozivi koji bi mogli blokirati thread su implementirani kao sistemski. Kada thread blokira, kernel može da izvršava ili drugi thread istog procesa ili thread drugog procesa. S obzirom na cijenu kreiranja i uništavanja threadova, neki sistemi pametno recikliraju svoje threadove. Ako je neki thread uništen, samo se označi kao nepodoban za izvršavanje, a ostali podaci o njemu ostanu netaknuti. Kasnije, taj stari thread se reaktivira. Kernel threadovi ne zahtijevaju nikakve nove, neblokirajuće sistemske pozive. Isto, ako jedan thread uzrokuje page fault, kernel može da razriješi problem. Glavni nedostatak ovih threadova su značajni troškovi sistemskih poziva, pa će biti veliki ukoliko su operacije s threadovima česte. Takođe, kernel threadovi ne rješavaju neke probleme. Šta se desi grananjem višenitnog procesa? Da li broj threadova ostaje isti ili ne? Izbor zavisi od narednih koraka. Drugi problem su signali – oni se šalju procesima, a ne threadovima, pa šta se dešava kad stigne signal, koji thread ga obrađuje? Moglo bi se učiniti da svaki thread identifikuje interes za određene signale, ali šta ako dva threada pokazuju interes za isti signal? Ovo su samo neki problemi.

Hibridne implementacije bi trebalo da kombinuju prednosti korisničkih threadova sa prednostima kernel threadova. Jedan način je da se koriste kernel threadovi i da se onda na neke ili sve njih multipleksiraju korisnički threadovi. Kod ovog pristupa programer može da odredi koliko kernel threadova da se koristi, a koliko korisničkih da se multipleksira na svaki. Primjenom ovoga, kernel je svjestan samo kernel threadova i raspoređuje samo njih. Oni ostali se kreiraju, uništavaju i raspoređuju baš kao obični korisnički threadovi.

Aktivacije raspoređivača: Iako su kernel niti poprilično bolje od korisničkih, neosporno su sporije. Tražio se način da se poboljša situacija bez gubitka dobrih karakteristika. Cilj rada na aktivaciji raspoređivača je da oponaša funkcionalnosti kernel threadova, ali sa boljim performansama i većom fleksibilnošću koja se povezuje sa paketima threadova implementiranim u korisničkom prostoru. Korisnički threadovi ne bi trebalo da moraju da prave specijalne neblokirajuće pozive ili vrše provjeru da li mogu izvršiti poziv sigurno. Ukoliko thread blokira ili dovede do page faulta, trebalo bi moći izvršavati ostale threadove unutar istog procesa. Efikasnost se postiže izbjegavanjem nepotrebnih prelaza između korisničkog i kernel prostora. Ako thread blokira jer čeka drugi thread, nema potrebe da se u to uključuje kernel. Kada se koriste aktivacije raspoređivača, kernel dodjeljuje određen broj virtuelnih procesora svakom procesu i prepušta korisničkom prostoru alociranje threadova procesoru. Ovo se može primjeniti i kod multiprocesora. Kernel može da uzme nazad dodijeljeni virtuelni procesor da bi ga dodijelio halapljivijem procesu. Kada kernel zna da je neki thread blokirao, obavještava run-time sistem procesa. Aktiviranjem, run-time sistem može prerasporediti threadove. Nakon što kernel sazna da prethodno blokirani thread više nije blokirao, šalje notifikaciju run-time

sistemu. Ova obavještenja se nazivaju **upcalls**. Run-time sistem označava taj thread kao ready. Kada se javi hardverski prekid dok se izvršava korisnički thread, prekinuti CPU se prebacuje u kernel mod. Ako prekid nije od interesa prekinutom procesu, prekinuti thread se vraća u stanje u kom je bio prije prekida. Ako jeste od interesa, prekinuti thread ne biva restartovan. Umjesto toga, on je suspendovan. Na run-time sistemu je da donese odluku da li će na novom virtuelnom procesoru nastaviti taj thread ili neki novi. Dakle, upcall mehanizam je od ključnog značaja ovdje – inače, sloj n nudi usluge koje sloj n+1 može da poziva, ali obrnuto ne važi. Upcalls ne prate ovaj princip.

Pop-Up threadovi su često korisni u distribuiranim sistemima. Važan primjer je obrada poruka koje pristižu (npr. zahtjeva za uslugom). Tradicionalni pristup je da postoji proces ili thread koji je blokiran na sistemu za primanje poziva dok čeka na dolaznu poruku. Kada poruka stigne, on je prihvata, raspakuje, izuča sadržaj i izvrši obradu. No, moguć je potpuno drugačiji pristup u kojem pristizanje poruke uzrokuje da sistem kreira novi thread koji će je obraditi. Takav thread naziva se pop – up thread. Ključna prednost pop-up threadova je da su potpuno novi, nemaju nikakvu istoriju – registre, stek.. Svaki počinje ispočetka i svi su ispočetka identični. Ovo omogućava brzo kreiranje threadova. Novom threadu se daje na obradu pristigla poruka. Rezultat upotrebe pop-up threadova je veoma kratko kašnjenje od stizanja poruke do započinjanja procesa njene obrade. Potrebna su neka planiranja unaprijed prilikom upotrebe ovih threadova. Npr, u kojem procesu se izvršava thread? Ako je moguće u prostoru kernela, to je jednostavnije i brže nego u korisničkom prostoru. To olakšava i pristup svim tabelama kernela i I/O uređajima. S druge strane, thread pun grešaka u kernelu može da napravi više štete nego takav u korisničkom prostoru. Ako se predugo izvršava i ne postoji način da bude prekinut, ulazni podaci mogu biti trajno izgubljeni.